# ANALYSIS AND DEMONSTRATION OF HTTP REQUEST SMUGGLING ATTACKS IN WEB APPLICATIONS

Submitted in partial fulfillment of the requirements for the Security Project

Submitted by:
Purv Doshi - 16014024016 - EXCP
Hardik Bhutada - 16014024008 - EXCP
Tanmay Jarande - 16014024024 - EXCP

Submitted to:
Mr. Zaheed Shaikh

Academic Year: 2026–2027
Submission Date: 18 October 2026

# ABSTRACT

HTTP Request Smuggling is a web security vulnerability that occurs when front-end and back-end servers interpret the boundaries of HTTP requests differently. This disagreement can cause parts of one request to be interpreted as another request, potentially leading to security issues in web applications.

This project focuses on understanding, analyzing, and demonstrating HTTP Request Smuggling using PortSwigger Web Security Academy and Burp Suite Community Edition. The practical work has used a controlled, intentionally vulnerable training environment to examine request parsing differences, study the behavior of HTTP headers, and observe the responses produced during testing.

The project has also examined preventive measures, including consistent HTTP request parsing, rejection of ambiguous requests, and secure server configuration. Screenshots and experimental observations have been documented to support the analysis.


# 1. INTRODUCTION

Modern web applications commonly use multiple servers and intermediary components to process HTTP requests. These components must interpret request boundaries consistently to ensure secure communication.

HTTP Request Smuggling can occur when different components disagree about how a request should be interpreted. Studying this vulnerability helps explain how HTTP protocol handling errors can affect web application security.

This project uses PortSwigger Web Security Academy's request-smuggling training labs and Burp Suite to study the vulnerability in a controlled environment.

# 2. PROBLEM STATEMENT

Different HTTP-processing components may interpret request boundaries inconsistently, creating opportunities for HTTP Request Smuggling vulnerabilities. This project investigates these parsing differences through controlled practical experiments and examines their security implications and possible mitigation techniques.

# 3. OBJECTIVES

1. To understand the concept, working principles, and security implications of HTTP Request Smuggling.
2. To study discrepancies in HTTP request parsing, particularly those involving Content-Length and Transfer-Encoding headers.

3. To use Burp Suite to inspect HTTP requests and responses in an authorized PortSwigger training environment.
4. To demonstrate and document a suitable request-smuggling lab, recording the actual observations and results.
5. To identify recommended prevention and mitigation techniques for HTTP Request Smuggling vulnerabilities.

# 4. SCOPE OF THE PROJECT

The project covers the theoretical study and practical demonstration of HTTP Request Smuggling using intentionally vulnerable web security training labs. It focuses on request analysis, controlled testing, interpretation of results, and defensive recommendations.

Testing will be restricted to the designated training environment. The project does not involve testing unauthorized websites or production systems.

# 5. TOOLS AND TECHNOLOGIES

- **PortSwigger Web Security Academy:** Provides the controlled training environment and practical labs.
- **Burp Suite Community Edition:** Used to inspect and analyze HTTP requests and responses.
- **Burp Suite built-in Chromium browser:** Used to access the training lab through Burp Suite.
- **Microsoft Word:** Used to prepare the project report.
- **Windows:** Operating system used for the practical setup.

# 6. METHODOLOGY AND LAB SETUP

The practical work used the PortSwigger Web Security Academy lab “HTTP request smuggling, basic CL.TE vulnerability” and Burp Suite Community Edition on Windows. The lab was opened in the browser and its requests were observed in Proxy > HTTP history. A baseline request was sent to Repeater. Because the intended technique requires HTTP/1.1, the Repeater request protocol was changed through Inspector > Request attributes > Protocol. The request was then crafted with Content-Length: 6 and Transfer-Encoding: chunked and sent twice, as specified by the lab instructions. All testing was limited to the authorized training lab.




**Figure 1. PortSwigger Web Security Academy lab page loaded in the browser**

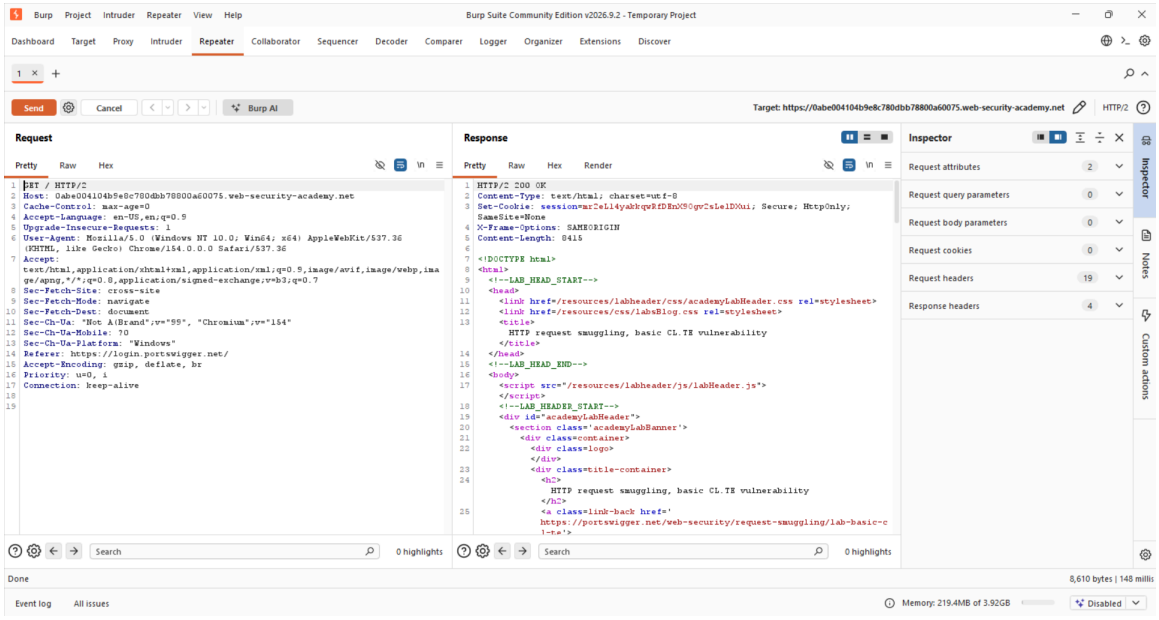



**Figure 2. Baseline request and response inspected in Burp Suite**

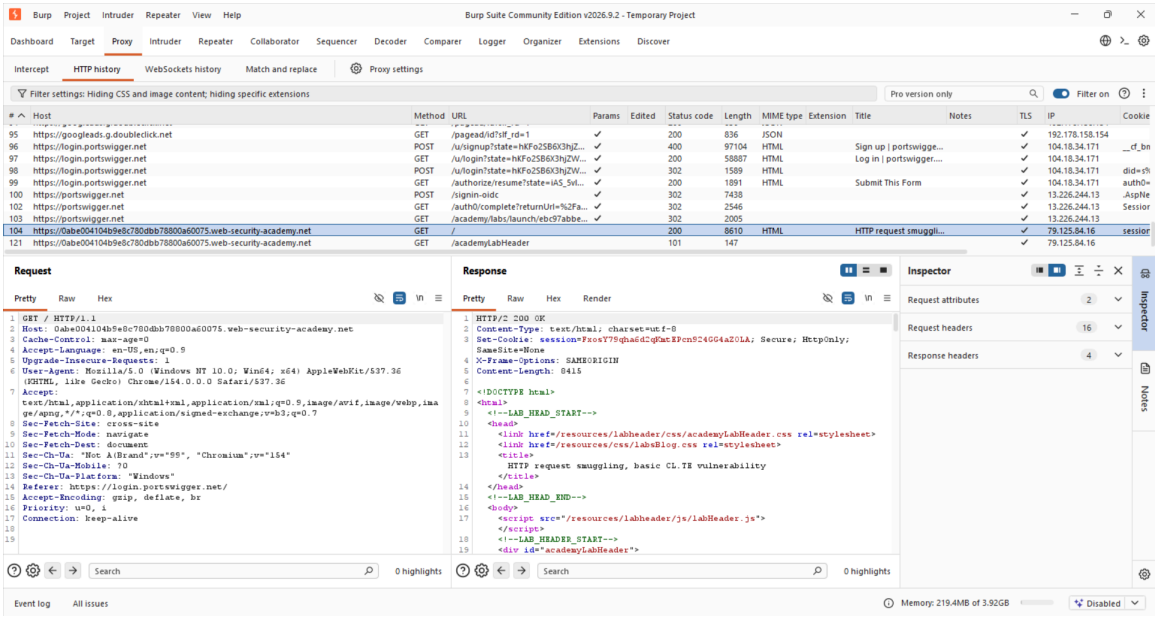



**Figure 3. Lab request visible in Burp Proxy HTTP history**

# 7. PRACTICAL IMPLEMENTATION AND EXPERIMENTS

The following experiment records the steps actually performed and the evidence captured. The lab completion status was verified on the PortSwigger lab description page.

## Experiment 1: Baseline HTTP request inspection

Aim: Establish a baseline by accessing the lab and inspecting a normal HTTP request and response in Burp Suite.

Procedure: Opened the lab in the browser, observed the request in Proxy > HTTP history, and reviewed the corresponding request and response.

Observation and result: The baseline GET request returned HTTP 200 OK and the lab page content. This provided a reference point before sending the crafted request.

Evidence: Figures 1–3 show the lab page, baseline request/response, and HTTP history.

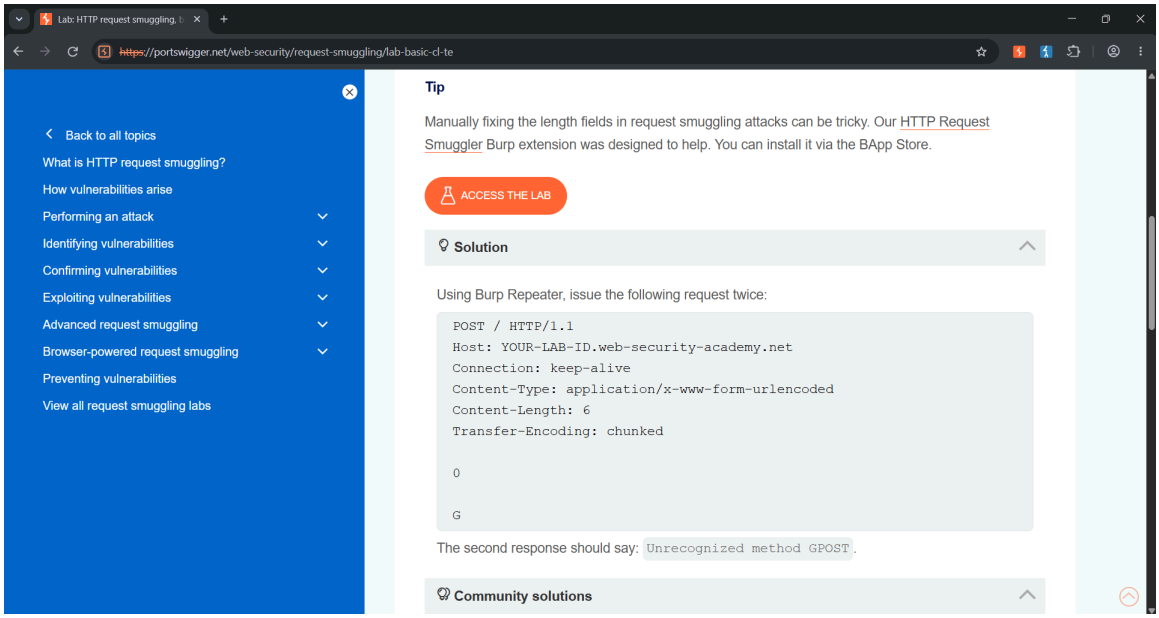



**Figure 4. PortSwigger's documented Basic CL.TE solution and expected response**

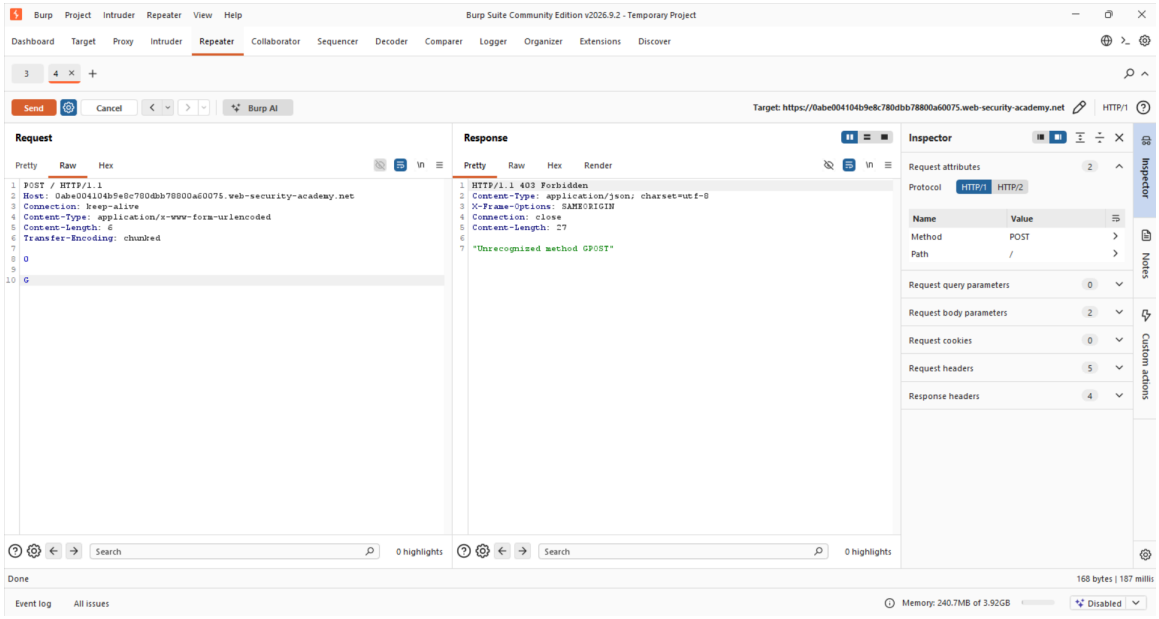



**Figure 5. Burp Repeater response: HTTP/1.1 403 Forbidden -"Unrecognized method GPOST"**

## Experiment 2: Request-smuggling lab selected for demonstration

Aim: Demonstrate the Basic CL.TE request-smuggling behavior in the authorized training lab.
Procedure: In Repeater, selected HTTP/1.1 through Inspector > Request attributes > Protocol. Sent a POST / request with Connection: keep-alive, Content-Length: 6, Transfer-Encoding: chunked, and a body containing a zero-length chunk followed by G. The same request was sent twice.

Observation and result: The response displayed HTTP/1.1 403 Forbidden with the message "Unrecognized method GPOST." This is the expected indicator that the backend interpreted the smuggled bytes as part of the next request method.
Evidence: Figures 4 and 5 show the lab's published solution instructions and the observed Repeater response.

### Experiment 3: Analysis of observed behavior and security implications

Aim: Verify the outcome and relate the observed behavior to request-framing differences.
Procedure: Compared the Repeater response with the lab's stated expected result, then returned to the PortSwigger lab description page to check the completion status.
Observation and result: The PortSwigger lab description displayed "Solved." The result demonstrates the intended behavior in this specific lab; it does not establish that an unrelated production system is vulnerable.
Evidence: Figure 6 records the platform's solved status.

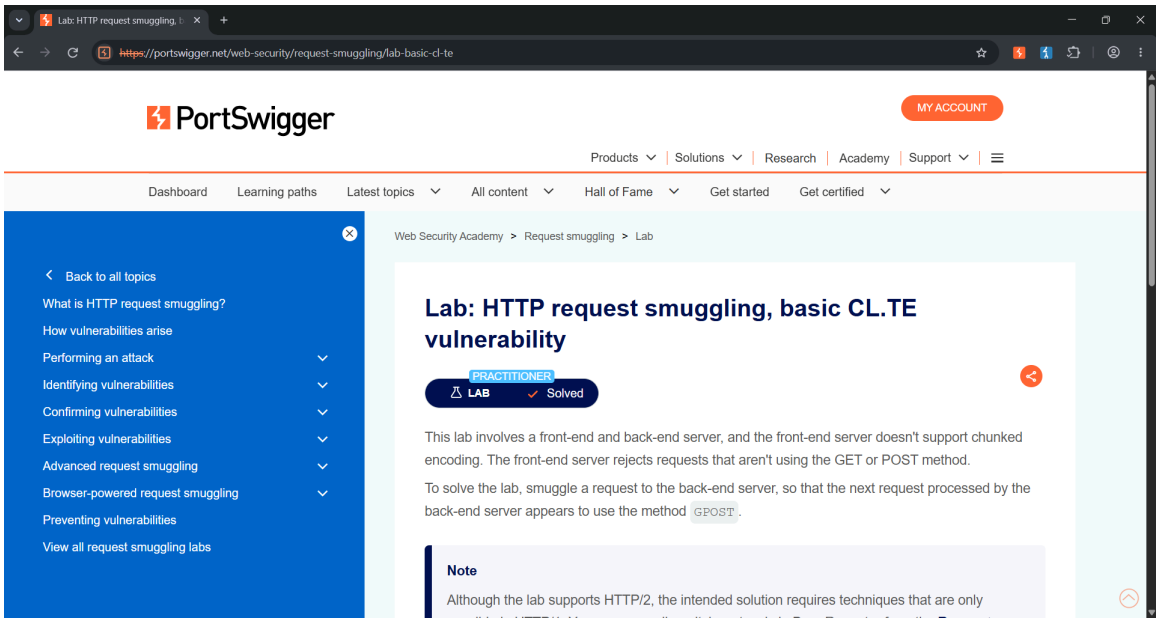



**Figure 6. PortSwigger Web Security Academy confirms that the lab is solved**

## 8. RESULTS AND ANALYSIS

The baseline GET request returned HTTP 200 OK. In the Basic CL.TE exercise, the HTTP/1.1 request used both Content-Length: 6 and Transfer-Encoding: chunked. The response "Unrecognized method GPOST" matched the lab's expected outcome, and the platform subsequently displayed "Solved." These observations demonstrate the lab's intended request-parsing discrepancy in a controlled environment. The result is specific to this training lab and should not be generalized to other systems without authorized testing.

# 9. DETECTION AND MITIGATION

Recommended defensive measures include ensuring that front-end and back-end servers parse HTTP requests consistently, rejecting ambiguous or malformed requests, keeping server software and intermediaries updated, and testing reverse-proxy and back-end configurations together.

Security monitoring should look for malformed or conflicting request-framing headers and unusual request/response patterns. Defensive controls should be validated against the specific server architecture in use.

# 10. CONCLUSION AND FUTURE SCOPE

The project demonstrated how disagreement over HTTP request framing can cause a back-end server to interpret request data differently from the front-end server. Using Burp Suite Repeater and HTTP/1.1, the Basic CL.TE lab produced the expected "Unrecognized method GPOST" response and was confirmed solved by PortSwigger. The screenshots provide evidence of the baseline request, crafted request, response, and completion status. Future work can compare additional request-smuggling variants and investigate defensive configuration approaches in authorized labs.

# 11. REFERENCES


1. PortSwigger Web Security Academy, "HTTP request smuggling." https://portswigger.net/web-security/request-smuggling
2. PortSwigger, "Burp Suite Documentation." https://portswigger.net/burp/documentation
3. OWASP Foundation, "OWASP Web Security Testing Guide." https://owasp.org/www-project-web-security-testing-guide/


# 12. APPENDIX: SCREENSHOT AND DELIVERABLE CHECKLIST

- S01 - Burp Suite opened successfully
- S02 - PortSwigger lab description
- S03 - Lab loaded in Burp's built-in browser
- S04 - Normal lab request in HTTP history
- S05 onward - Actual experiment requests and responses
- Final evidence - PortSwigger lab solved status
- Final deliverable - 7–8 minute demonstration video with participation of all group members, as required